Scale — Kernel Inventory

Purpose. This is the baseline for the Metaobject Protocol direction recorded in D16 and D17. It answers one question: *which language mechanisms are still host code, and what would it take to make them image-resident metaobjects with a protocol?* It is a snapshot, not a promise. A mechanism moves from the kernel to the protocol only when a stage's acceptance test passes.

Snapshot. Taken against the tree that records D15 (W4 unification) and updated for D50 (conditions and restarts). The suite at this state is 1118 checks (1083 examples and 35 properties). The numbers below are the baseline, not a permanent claim.

1. What counts as the kernel

The **kernel** is the F# code that must exist before a metaobject can be interpreted. It is the fixed point of the protocol:

- tensor construction and structural equality (`Tensor`, `tensorsEqual`);
- image append and lookup, text interning, and `EntryIdentity` allocation (`Image`);
- root threading (`Root`, `VocabularyRoot`, `UserWordsRoot`, `BindingsRoot`, `SetsRoot`, `DeclarationsRoot`, `GenericFunctionsRoot`);
- stack push, pop, and peek (`Stack`);
- the literal cases in `executeWord` (numeric, `:Name`, `?`, text) and the protocol dispatcher itself;
- the top-level `run` loop, reduction accounting, and source-position attachment.

Everything else is a **candidate**: if changing its behavior requires editing F#, it is a candidate for refinement.

2. The dispatcher (`components/prelude/evaluator/evaluator.fs`)

`executeWord` has **91 explicit word cases** at D59, plus seven token classes and four lookup layers. (The earlier headline tracked only the newer families; this is the corrected full dispatcher count.)

Family	Count	Words	Current form	Metaobject candidate?
Sigils	3	`{`, `[`, `]`	host literal and collector	`{` is a control word (D51); `[` and `]` remain sigils
Arithmetic and comparison	7	`add`, `subtract`, `multiply`, `divide`, `lessThan`, `greaterThan`, `equals`	`binaryOperation` closures; elementwise over equal shapes, a scalar is rank 0, a mismatch is `ShapeMismatch` (D60)	wrapper word; the primitive stays in the kernel
Stack shuffling	5	`duplicate`, `swap`, `drop`, `over`, `rotate`	`EvaluatorStack`	wrapper word; the primitive stays in the kernel
Control	3	`if`, `apply`, `define`	text splicing with control-word dispatch (D51)	**landed (D51)** the defaults remain kernel primitives

Family	Count	Words	Current form	Metaobject candidate?
Control protocol	5	`control`, `control-quotation`, `control-if`, `control-apply`, `control-define`	image-resident word entries; default primitives run in the caller frame, user handlers are code cells (D51)	**landed (D51)** a new form needs no F#
Output	1	`print`	`Output` append	effect protocol, low priority
Bookmarks	4	`bookmark`, `jump`, `copyBookmark`, `showBookmark`	tensor layout `[Location; Baseline; Position; stepId]`; a bookmark names an event-stream step (D29)	absorbed by the event-stream (D29)
Event-stream	2	`history`, `rollback`	image-resident, append-only steps; rollback restores a step	**landed (W8/ D29)**
Ownership	6	`own`, `borrow`, `consume`, `release`, `reclaimable`, `compact`	per-run execution-state map; named refusals; a conservative static reference-linearity report; a sparse successor with stable addresses	**landed (W9/ D30)** reclaim report is D52; compaction is D53; roots include every live actor (D82); epoch recording is D85
Ownership types	1	`linearity`	a usage mode (`ExactlyOnce`, `AtMostOnce`, `AnyNumberOfTimes`) over a trace; a named language of traces; a tagged verdict tensor	**landed (D54)** modes and trace linear types are D81; undecided is named, never a silent pass
Scheduler	6	`spawn`, `yield`, `halt`, `actors`, `resume`, `kill`	per-process scheduler state; actor snapshots	**landed (W5/ D22)**
Channels	4	`channel`, `send`, `receive`, `close`	image-resident identity and appended state (D27)	**landed (W6/ D20)**
Events	7	`send-event`, `receive-event`, `choose`, `wrap`, `guard`, `with-fallback`, `sync`	image-resident event tensors	**landed (W7/ D26)**
Word metaobject	1	`word`	image-resident word definition	**landed (D21)**
Guards	2	`guard-define`, `guard-holds`	image-resident guard definitions; the shared	**landed (D35)**; the lattice

Family	Count	Words	Current form	Metaobject candidate?
			probe; a document refinement may name one through the lattice resolver	bridge is D41; the six typed SMT projections remain host guards
Environment and terms	9	`bind`, `lookup`, `unify`, `unify-verdict`, `resolve`, `term`, `typed`, `lambda-term`, `reduce`	F# algorithms; `TermTag = -2`, `TypedTag = -3`, `LambdaTag = -15`, `UnificationVerdictTag = -27`; Miller-pattern higher-order unification (D59)	stage 3/4: sort-aware term protocol; the unifier can remain a default method
Set algebra	6	`set`, `add-member`, `and`, `or`, `nor`, `is`	finite member lists	stage 3
Sorts	16	`seal`, `predicate-seal`, `constructor`, `constructor-seal`, `unname`, `subset`, `filter`, `union`, `intersect`, `except`, `member`, `same-shape`, `same-name`, `sort-of`, `merge`, `complement`	finite extensions, image-resident predicates (D31), named constructors (D36), verdict-aware filtering (D34), and image-resident operation methods (D39); the `set-*` primitives remain host defaults	stage 3: predicate sorts, filtering, named constructors, and the six set-operation protocol methods landed (D39); the operation primitives are still F# bodies
Dispatch	2	`generic`, `method`	image-resident generics and methods; type-first selection, exact-tuple fast path, extension specificity	**landed (D19/W10)**; a new method needs no F#
Classification	1	`class-of`	total verdict over entry kind and the unique most-specific sort (D32)	**landed (D32)**; the query is the kernel primitive
Total	**91**			

The explicit cases are the quoted tokens in `executeWord`. The four lookup layers are the fallback chain: built-in match (control words are checked before the built-in token table), then `UserWordsRoot`, then `GenericFunctionsRoot`, then `VocabularyInImage.lookupByToken`. That order is host policy; a user word cannot shadow a built-in, a generic function cannot shadow a built-in, and a user word can shadow a generic function. D17 makes the order a lookup protocol so the policy is explicit and overridable.

Token classes (not counted as word cases):

Class	Where	What is host
numeric literal	`System.Double.TryParse`	number parsing, `Tensor.scalar`
`:Name`	`token.StartsWith(":")`	sigil split, `Image.addText`
`?name`	`token.StartsWith("?")`	variable sigil
text literal	`isTextLiteral`	quote stripping
user word	`findPair` on `UserWordsRoot`	latest-write-wins lookup, body splice
vocabulary word	`VocabularyInImage.lookupByToken`	token scan, `IsActive`
unknown	terminal `None`	`UnknownWord`

Freeze at D68. The explicit-case count is now 135: D59′s 91 plus the actor model, monitors, registry, selective receive, and the other families added since. The table above is the D59 baseline, not an exhaustive current list; the count is the guard. The freeze rule is: a new explicit case requires an inventory update and a decision record in the same change. The alternative is an image-resident word or generic. No new F# case is added silently.

3. Host mechanisms outside the dispatcher

Mechanism	File	Host-resident behavior	Metaobject or protocol candidate
Sigil splitting and multi-word combination	`tokenizer.fs`	`[`, `]`, and `:` are hardcoded; vocabulary forms combine one to three tokens	read table or token protocol (separate decision)
Sentence recognition	`parser.fs`	precedence: Conditional, then Negation, then Quantifier before Function Name; subject, conditions, and verb fallbacks	sentence-pattern protocol or decision table (product boundary)
Vocabulary entry	`types.fs`, `vocabulary.fs`, `vocabulary-in-image.fs`	`FsharpName` is a host seam; category is a float; `IsActive` is a meta-level flag	**stage 2**: `Word` metaobject with an implementation reference
Emission	`emitter.fs`	`DomainBinding` is data (D9), but `emitFSharp` and `emitSmt` are host renderers	artifact renderer protocol (product layer; out of near-term scope)
SMT encoding	`smt.fs`, `domain-expression.fs`	`createDomain`, `guardToBooleanExpression`, `satisfiable`, `discharge`; only boolean and integer sorts	**stage 3/4**: sort and guard encoder protocol, `Undecided` default
Guard evaluation	`type-lattice.fs`, `components/prelude/evaluator/runtime-guard.fs`, `refiner.fs`	the lattice evaluates a named guard through `EvaluatorRuntimeGuard.resolver`; the six typed relations remain a host pattern match	**stage 4**: the runtime half landed as D35 and the lattice half as D41; a named guard is executable but not SMT-encodable

Mechanism	File	Host-resident behavior	Metaobject or protocol candidate
Candidate synthesis	`refiner.fs`	`candidates` enumerates thresholds from a finite universe	domain policy; not a protocol requirement
Decision table	`decision-table.fs`	`select` and `check`; `Unique`/`First`/`Priority`; `Selected`/`NoMatch`/`Ambiguous`; `Sound`/`Unsound`/`Undecided`	**stage 4 prototype**: method applicability, specificity, and ambiguity
Fixed-spec acceptance	`fixed-specification-acceptance.fs`	`FrozenSpecification`, `ImplementationProposal`, `SpecificationProposal`, `SpecChangeCapability`, `accept`, bundle and corpus JSON	**stage 6**: typed proposals, D79
Nearest sentence	`nearest-sentence.fs`	`candidates`, `nearest`, `nearestWith`, `advice`	**stage 6**: diagnostic advice, not a proposal, D71/D79
Persistence	`persistence.fs`	binary format; roots are not serialized, only image data	kernel format; a root table is a follow-on
Cell-system encoding, closures, classes/slots/instances, precedence/dispatch, method combination, metaclasses, conditions/restarts	`components/prelude/evaluator/cell-system.fs`, `components/prelude/evaluator/closure.fs`, `components/prelude/evaluator/class-system.fs`, `components/prelude/evaluator/condition.fs`, `components/prelude/evaluator/dispatch.fs`	tags, tensor codecs, class and instance-state roots, lexical frames, `let`/`lambda`/`apply`, `class`/`slot`/`create-instance`/`slot-value`/`set-slot-value`, C3 precedence, class specializers, method qualifiers and combination, metaclasses and `cell-class`, conditions and restarts	**D44–D50** complete
Scheduler	`scheduler.fs`	per-process state threaded through `EvaluationState`; token budgets, supervision, mailboxes, links, monitors, registry, and selective receive	**landed (W5, D63–D68)** the design sketch was replaced

4. Classification

Keep in the kernel

- `Tensor`, `Image`, `Lexicon`, `Stack`, and `EntryIdentity`.
- Text interning and `Image.addText`/`readText`.
- Root threading and persistence of image data.
- Numeric, name, and text literals.
- The `run` loop, reductions, and source positions.
- The protocol dispatcher itself: the one match that finds a metaobject and applies it.

Candidates, in dependency order

1. **Word metaobject** (D17 stage 2). `VocabularyEntry` becomes a projection of `[WordTag; token; form; category; active; implementation]`. The implementation is an image reference, not an `FsharpName`. Acceptance: a new word family needs no F#.
2. **ValueSet metaobject** (stage 3, constructor half landed as D36). A sort name carries a narrowing implementation: a raw predicate quotation (D31) or a named image-resident constructor (D36). `member` and `subset` resolve it through the shared probe. The set operations remain kernel defaults and refuse a constructor sort rather than approximate it. Remaining acceptance: `member`, `subset`, `union`, `intersect`, `except`, and `complement` become protocol methods, and a new composite constructor needs no F#.
3. **Generic function and method** (stage 4, **partly landed as D19/W10**). A generic function is `[GenericTag; name; arity; policy; method…]`; a method is `[MethodTag; arity; body; specializer…]`. Dispatch is type-first over the top N stack operands, with an exact-tuple fast path and extension-subset specificity; ambiguity is `AmbiguousDispatch`. Acceptance: a new method needs no F# (met); no silent ambiguity (met). The policy surface and a resolved method cache remain follow-on work.
4. **Guard evaluator** (stage 4, runtime half landed as D35, lattice half as D41). A guard is an image-resident quotation and the lattice resolves a named guard through the evaluator, so a new guard needs no F# at the evaluation seam. A named guard is executable and generative but has no SMT encoder; Z3 reports `Unavailable` rather than approximating it. The six typed relations remain host guards with host encoders.
5. **Protocol failures** (stage 6). `NoApplicableMethod`, `NoNextMethod`, `ValueSetNotClosed`, `Ambiguous`, and `Undecided` become values or witnesses, consistent with E1 and `Repair`.

Out of scope for this direction

- A reflective tower: the evaluator itself as a metaobject, `eval` as a protocol.
- A grammar protocol for the tokenizer and parser. That is a separate surface decision; the product layer is not the bench.
- The scheduler and concurrency (Track C).
- Exposing metaobjects to the child-facing sentence layer. D9′s domain binding remains the surface.
- The optional DSPy-shaped model layer (D42). It is a host-side product slice: a signature projects from the document and the lattice, the model call is an edge effect, and the optimizer is host code. It adds no kernel case and no word, and nothing on the production path depends on it.

5. Stages and acceptance

Stage	Work	Acceptance (falsifiable)
0	This inventory	Every explicit word case and host mechanism is listed; a new case without an inventory update is a review failure.

Stage	Work	Acceptance (falsifiable)
1	Metaobject encoding (D16)	Round-trip; tag discrimination; address identity; no host metaobject store.
2	Word metaobject	A new word family is added without editing F#; the built-in shadowing policy is explicit.
3	ValueSet metaobject	A new constructor is added without editing F# (constructor half, D36); `member` and `subset` return a verdict, not a silent false; predicate and constructor sorts report `Undecided`. The six set-operation methods are image-resident and replaceable (D39); their default bodies are kernel primitives.
4	Generic functions and methods	A new method is added without editing F#; ambiguity is `Ambiguous` or `NoMatch`, never a silent pick. **Landed as D19 (W10): a method is added with `generic`/`method`; ambiguity is `AmbiguousDispatch`. The guard half is D35 (runtime) and D41 (lattice bridge): the image evaluates a named guard, and an unencodable guard is refused by name rather than dropped.**
5	Control refinement	A new control form is added without editing F#; quotation, `if`, `apply`, and `define` are protocol methods. **Landed as D51: control words are image-resident `Word` metaobjects; `handler name control` installs a code cell, and a later declaration replaces a form.**
6	Protocol failures	A protocol failure is a value or witness; no exception; `Repair` can consume it. **Landed as D50: conditions and restarts are image-resident values; handler frames are execution state, never serialized.**
7	Full protocol	Metaclasses, `cell-class` closure, and method combination. Encoding, closures, classes/slots/instances, and precedence/dispatch landed as D44–D48; metaclasses and conditions/restarts landed as D49–D50. Every D17 stage is now applied.

A stage is not done when the code compiles; it is done when the acceptance test passes.

6. What shrinking the kernel means, concretely

- Move a word family from the `executeWord` match to a `Word` metaobject.
- Move a sort operation from an F# function to a protocol method.
- Move a guard evaluator from `Refiner.evaluateGuard` to a protocol method.
- Move a protocol failure from an `EvaluationError` case to a value the protocol returns.

Each move keeps the suite green and adds the stage's acceptance test. The count in section 2 is the baseline: **80 explicit cases at D36**. A commit that adds a case without moving one is kernel growth, and needs a recorded reason.

7. Risks

- **The count is a proxy, not a proof.** A word can be a metaobject while its implementation is still an F# closure. The seam moves; it does not disappear. The acceptance test, not the count, is the real measure.
- **The kernel can grow silently.** A new built-in is easier than a new metaobject. The inventory is the counterweight.
- **Tags and roots.** D15 uses `-2` and `-3`; D19 adds `-4` (generic function) and `-5` (method); D36 adds `-14` (sort constructor). D16's tags are allocated append-only. New roots are execution state (D4) and must be re-threaded across persistence; a root table is a follow-on.
- **Performance.** Protocol dispatch is slower than a match; cache resolved words and methods, and keep the primitive operations in the kernel. D19's exact-specializer-tuple fast path is the first such cache: under `MostSpecific`, a typed or uniquely inferred operand tuple is selected by sort name without touching the extensions.
- **Determinism.** Protocol decisions that use SMT must record the solver version and return `Undecided` rather than a silent false. The existing `Smt.Outcome` and `DecisionTable.Verdict` are the model.
- **Scope.** The product layer (parser and emitter) and the concurrency track are not the bench. Do not let the protocol rewrite them by accident.